

APLICACIONES WEB • UNIDAD II • SEMANA 5

Cómo construir tu aplicación web en PERL

Guía paso a paso: de la instalación al navegador

Strawberry Perl • IntelliJ IDEA • HTML5 semántico • código explicado

Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D. • UTEQ

¿Qué vamos a lograr?

Una aplicación web servida por PERL

- Una página que saluda con «¡Hola Mundo!» y el nombre de la persona.
- El HTML lo genera un programa PERL en el servidor; no es un archivo estático.
- El usuario escribe su nombre en un formulario y la página responde.
- Funciona sin instalar Apache: usa un mini servidor en PERL puro.
- Se ejecuta y se prueba desde IntelliJ IDEA con un clic.

¡Hola Mundo, Gleiston!

Esta página la genera un programa **PERL** en el servidor, no es un archivo HTML estático.

Servido por PERL en http://localhost:8080

Resultado final en el navegador

Herramientas que necesitas

1

Strawberry Perl

El intérprete de Perl para Windows (versión 5.42). Incluye compilador y módulos.

2

IntelliJ IDEA

El entorno de desarrollo (IDE) donde escribiremos y ejecutaremos el programa.

3

Plugin Perl

La extensión «Perl» (Camelcade) que da soporte de Perl a IntelliJ.

4

Navegador web

Chrome, Edge o Firefox para abrir y probar la aplicación.

Los 7 pasos del proceso

1

Instalar Strawberry Perl

2

Instalar el plugin Perl en IntelliJ

3

Configurar el intérprete (perl.exe)

4

Crear el proyecto Perl

5

Escribir y entender el programa

6

Ejecutar con el botón Run

7

Abrir y probar en el navegador

Instalar Strawberry Perl

1

Acciones

- Descarga Strawberry Perl 5.42 desde strawberryperl.com.
- Ejecuta el instalador y acepta la ruta por defecto: C:\Strawberry.
- Al terminar, abre una terminal (cmd) y escribe: perl -v.
- Debe mostrar v5.42.2: ¡Perl ya está instalado!

Verificación en la terminal

```
C:\> perl -v
```

```
This is perl 5, version 42, ...  
built for MSWin32-x64
```

Si ves la versión, la instalación fue correcta

Strawberry Perl incluye el compilador y muchos módulos (entre ellos CGI), así que no tendrás que instalarlos por separado.

Instalar el plugin Perl en IntelliJ

2

Acciones

- Abre IntelliJ IDEA y ve a Settings (Ctrl+Alt+S).
- Entra en la sección Plugins.
- En la pestaña Marketplace, busca «Perl».
- Instala el plugin Perl (Camelcade) y reinicia el IDE.

Si la red bloquea el Marketplace

- Descarga el plugin (.zip) desde plugins.jetbrains.com.
- Settings → Plugins →  → Install Plugin from Disk.
- Selecciona el .zip descargado y reinicia.

Configurar el intérprete: apunta a perl.exe

3

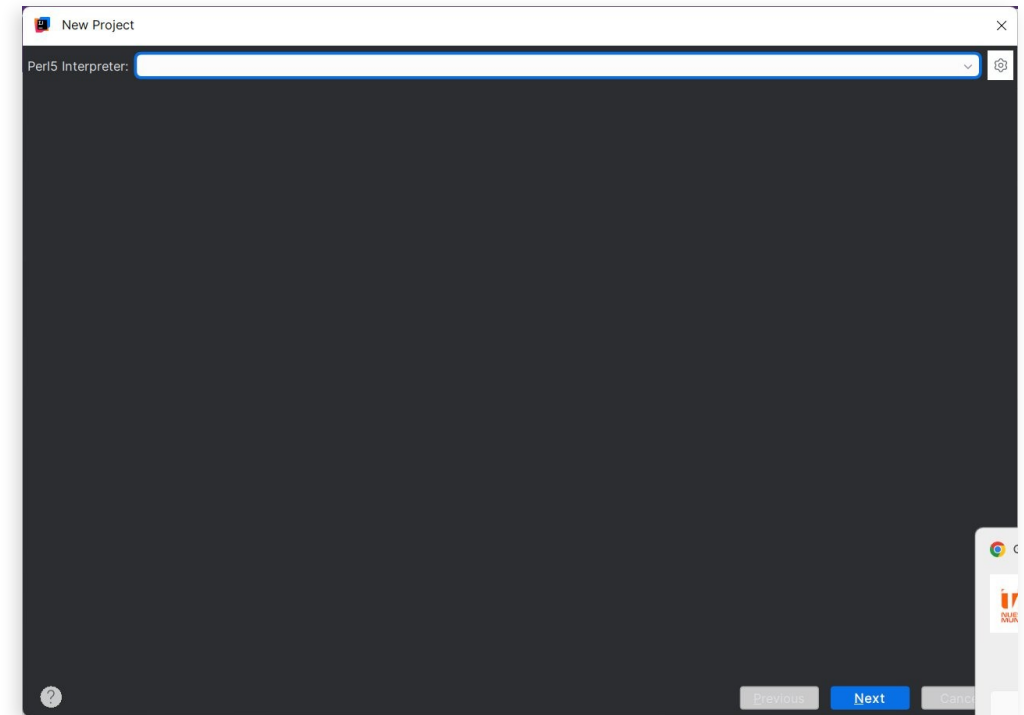
El archivo a buscar

perl.exe

Ruta por defecto:

`C:\Strawberry\perl\bin\perl.exe`

- Pulsa el engranaje ⚙️ junto al campo y elige examinar.
- Navega a esa carpeta y selecciona perl.exe.
- NO uses wperl.exe ni la carpeta c\bin (ahí está gcc).



Diálogo «Perl5 Interpreter» en IntelliJ

Crear el proyecto (y el aviso de certificado en la UTEQ)

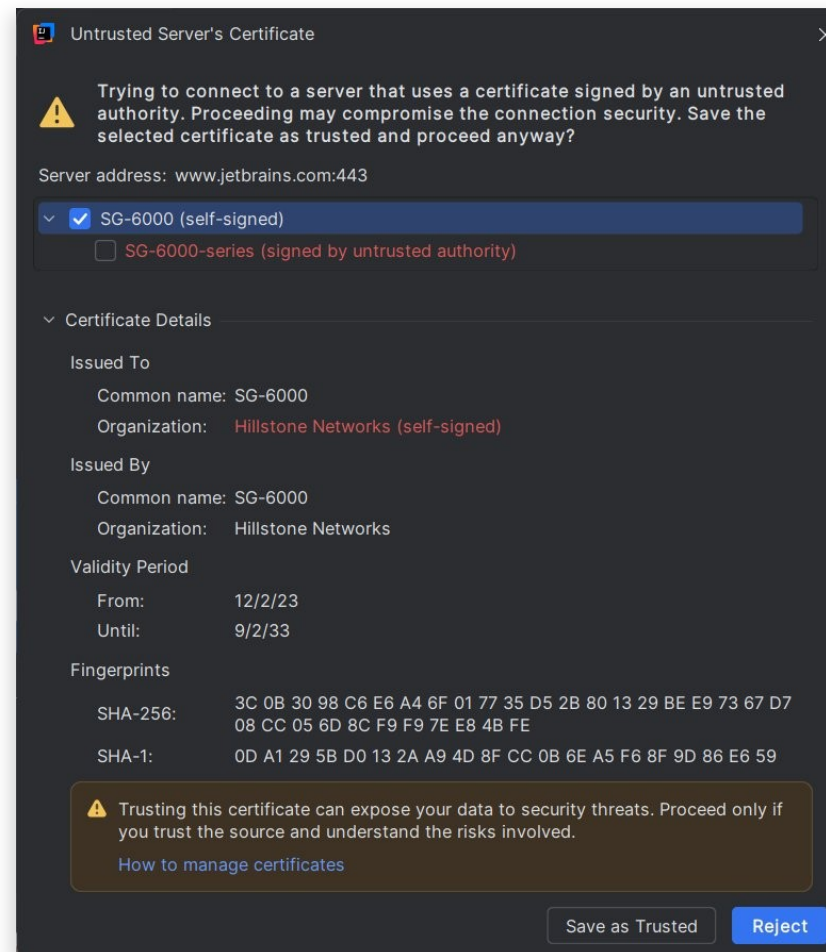
4

Crear el proyecto

- New Project → Perl → selecciona el intérprete → Next → Finish.
- Se crean las carpetas lib (módulos) y t (pruebas).

Si aparece «Untrusted Server's Certificate»

- Es el firewall de la UTEQ inspeccionando el tráfico (normal en la red institucional).
- En la red de la universidad: pulsa Save as Trusted para continuar.



Aviso del firewall (Hillstone) de la UTEQ

Escribir el programa: anatomía de hola_web.pl

5

Crear el archivo

- Clic derecho en el proyecto → New → File.
- Nómbralo hola_web.pl (en la raíz del proyecto).

Lo veremos línea por línea

En las siguientes diapositivas se explica CADA instrucción, operador y función del programa.

El programa en 8 bloques

- A** Pragmas, módulo y variable del puerto
- B** Crear el servidor (socket)
- C** Bucle de atención y lectura de la petición
- D** Lectura segura del nombre (expresión regular)
- E** Construir el HTML5 (heredoc)
- F** Enviar la respuesta HTTP
- G** Función url_decode (decodificar la URL)
- H** Función escape_html (defensa anti-XSS)

Pragmas, módulo y variable del puerto

```
use strict;           # exige declarar variables
use warnings;         # activa avisos utiles
use IO::Socket::INET; # modulo de sockets

my $puerto = 8080;    # puerto a escuchar
```

Qué significa cada elemento

— Inicia un comentario: Perl ignora todo lo que sigue hasta el fin de línea.

use — Carga en el programa una directiva (pragma) o un módulo.

strict / warnings — Pragmas: obligan a declarar variables y avisan de prácticas peligrosas.

IO::Socket::INET — Módulo del núcleo para conexiones de red; «::» separa los niveles del nombre.

my — Declara una variable local (lexical), visible solo en su bloque.

\$ — Símbolo de variable ESCALAR (un solo valor: número o texto).

= y ; — «=» asigna un valor; «;» termina toda instrucción.

Crear el servidor (socket)

```
my $servidor = IO::Socket::INET->new(  
    LocalAddr => '127.0.0.1', # solo este PC  
    LocalPort => $puerto,    # puerto 8080  
    Proto     => 'tcp',       # protocolo TCP  
    Listen    => 5,           # cola de espera  
    ReuseAddr => 1,           # reusar puerto  
) or die "No abrio $puerto: $!\n";
```

Qué significa cada elemento

- >** — Operador flecha: llama a un método de un objeto. Aquí «new» crea el socket.
- new** — Constructor: fabrica el objeto socket servidor.
- =>** — «Coma gorda»: une cada opción con su valor y entrecomilla el nombre de la izquierda.
- LocalAddr / LocalPort** — Dirección y puerto donde escucha (127.0.0.1 = este equipo).
- Proto / Listen / ReuseAddr** — Protocolo TCP, tamaño de la cola de conexiones y reutilización del puerto.
- or** — O lógico de baja prioridad: si «new» falla (valor falso), ejecuta lo siguiente.
- die** — Detiene el programa mostrando un mensaje de error.
- \$!** — Variable especial con el último error del sistema operativo.
- \n** — Secuencia de escape: salto de línea (válida en comillas dobles).

Bucle de atención y lectura de la petición

```
print "Escuchando en :$puerto\n";

while (my $cliente = $servidor->accept) {
    my $peticion = <$cliente>;
    $peticion = '' unless defined $peticion;
    # ... continua en el bloque D ...
}
```

Qué significa cada elemento

print — Escribe texto (aquí en la consola). En comillas dobles, «\$puerto» se interpola.

while (COND) { } — Repite el bloque mientras la condición sea verdadera.

\$servidor->accept — Espera (bloquea) hasta que llega una visita y devuelve un socket para ese navegador.

my \$cliente = ... — Guarda ese socket; la asignación dentro del while mantiene vivo el bucle.

<\$cliente> — Operador de lectura de línea: lee UNA línea enviada por el navegador.

unless — Lo contrario de «if»: ejecuta solo si la condición es FALSA (modificador al final).

defined — Indica si la variable tiene algún valor (no está indefinida).

'' — Cadena vacía (comillas simples: texto literal, sin interpolación).

Lectura segura del nombre (expresión regular)

```
my $nombre = 'Mundo'; # valor por defecto

if ($peticion =~ m{[?&]nombre=([^\s&]*)}) {
    $nombre = url_decode($1); # lo capturado
}
# escapar la salida (anti-XSS):
$nombre = escape_html($nombre);
```

Qué significa cada elemento

if (COND) { } — Ejecuta el bloque solo si la condición es verdadera.

=~ — Enlaza una variable con una expresión regular (la evalúa sobre ese texto).

m{ ... } — Operador de coincidencia (match); aquí usa llaves como delimitadores.

[?&] — Clase de caracteres: un «?» o un «&» (inicio del parámetro en la URL).

(...) — Grupo de captura: guarda lo que coincide dentro.

[^\s&]* — Clase NEGADA: cualquier carácter que NO sea espacio (\s) ni «&»; «*» = cero o más.

\$1 — Lo capturado por el primer par de paréntesis.

url_decode / escape_html — Llamadas a nuestras funciones (definidas en los bloques G y H).

Construir el HTML5 (heredoc)

```
my $html = <<"HTML";
<!DOCTYPE html>
<html lang="es-EC">
    ...
    <h1>Hola Mundo, $nombre!</h1>
    ...
    \@media (max-width:640px){ ... }
    info\@uteq.edu.ec
</html>
HTML
```

Qué significa cada elemento

my \$html = <<"HTML"; — Inicia un HEREDOC: guarda en \$html todo el texto hasta una línea que diga solo «HTML».

Comillas dobles en <<"HTML" — Hacen que Perl INTERPOLE las variables (\$nombre, \$puerto) dentro del texto.

Terminador HTML — Debe ir solo, al inicio de la línea, para cerrar el heredoc.

\@ y \\$ — Una «@» o «\$» literal (p. ej. \@media o el correo) se ESCAPAN para que Perl no las tome como variables.

¡ á ... — Entidades HTML: forma segura de escribir símbolos y tildes.

Enviar la respuesta HTTP

```
my $bytes = length $html; # tamaño cuerpo

print $cliente "HTTP/1.1 200 OK\r\n";
print $cliente
    "Content-Type: text/html; charset=utf-8\r\n";
print $cliente "Content-Length: $bytes\r\n";
print $cliente "Connection: close\r\n";
print $cliente "\r\n";    # línea en blanco
print $cliente $html;     # el cuerpo HTML

close $cliente;           # cierra conexión
```

Qué significa cada elemento

length — Devuelve cuántos caracteres tiene la cadena (aquí, el tamaño del cuerpo).

print \$cliente "..." — Con el filehandle \$cliente tras print, el texto va al NAVEGADOR (al socket), no a la consola.

\r\n — Retorno de carro + salto de línea: el fin de línea que exige el protocolo HTTP.

HTTP/1.1 200 OK — Línea de estado: versión del protocolo y código 200 (todo correcto).

Content-Type / -Length — Cabeceras: tipo de contenido (HTML, UTF-8) y tamaño del cuerpo.

Connection: close — Cabecera: la conexión se cierra al terminar la respuesta.

línea en blanco — Un \r\n vacío que separa obligatoriamente las cabeceras del cuerpo.

close — Cierra el socket del cliente y libera la conexión.

Función `url_decode` (decodificar la URL)

```
sub url_decode {  
    my ($s) = @_;          # 1er argumento  
    $s =~ tr/+/ /;         # '+' -> espacio  
    $s =~  
        s/%([0-9A-Fa-f]{2})/chr(hex($1))/ge;  
    return $s;             # ya decodificado  
}
```

Qué significa cada elemento

sub NOMBRE { } — Define una FUNCIÓN (subrutina) reutilizable.

@_ — Arreglo con los argumentos recibidos; «@» es el sigilo de los arreglos (listas).

my (\$s) = @_; — Copia el primer argumento en el escalar \$s (asignación de lista).

tr/+/ / — Transliteración: cambia caracteres uno a uno (cada «+» por un espacio).

s/PATRÓN/REEMPLAZO/ — Sustitución: reemplaza lo que coincide con el patrón.

g y e — «g» = todas las coincidencias; «e» = evalúa el reemplazo como código Perl.

[0-9A-Fa-f]{2} — Dos dígitos hexadecimales; «{2}» = exactamente dos.

hex / chr — «hex» pasa texto hexadecimal a número; «chr» da el carácter de ese código.

return — Devuelve el resultado a quien llamó la función.

Función `escape_html` (defensa anti-XSS)

```
sub escape_html {  
    my ($s) = @_;  
    $s =~ s/&/&amp;/g;    # PRIMERO el &  
    $s =~ s/</&lt;/g;    # < no abre etiqueta  
    $s =~ s/>/&gt;/g;    # > no cierra etiqueta  
    $s =~ s/"/&quot;/g;    # comilla segura  
    return $s;  
}
```

Qué significa cada elemento

`s/&/&/g` — Reemplaza TODOS los «&» por «&». Va PRIMERO para no re-escapar los demás.

`s/</</g` — Convierte «<» en «<» para que el navegador no lo lea como inicio de etiqueta.

`s/>/>/g` — Convierte «>» en «>» (fin de etiqueta).

`s/"/"/g` — Hace segura la comilla doble dentro de atributos HTML.

`g` (global) — Aplica el reemplazo a todas las apariciones, no solo la primera.

Conjunto — Defensa anti-XSS: convierte la entrada del usuario en texto inofensivo.

`return` — Devuelve el texto ya seguro para mostrarse en la página.

Tarjeta rápida: símbolos, operadores y funciones

Símbolos y operadores

\$ / @ — Sigilo de escalar (un valor) / de arreglo (lista).
; **=** — Fin de instrucción / asignación.
-> => — Llamada a método / «coma gorda» (opción => valor).
=~ — Enlaza un texto con una expresión regular.
m{} **s///** **tr///** — Coincidir / sustituir / transliterar.
g e — Modificadores: global / evaluar como código.
() [] [^] \s * {2} — Captura / clase / clase negada / espacio / repetición.
<\$fh> \$1 \$! — Leer línea / 1.ª captura / último error del sistema.
\n \r\n <<" — Salto de línea / fin de línea HTTP / heredoc.

Funciones y palabras clave

use — Carga un módulo o pragma.
my — Declara una variable local.
if / unless — Ejecuta si es verdadero / si es falso.
while — Repite mientras sea verdadero.
defined — ¿La variable tiene valor?
print — Escribe (consola o socket según el filehandle).
die — Aborta el programa con un mensaje.
length — Número de caracteres de una cadena.
chr / hex — Carácter de un código / hex a número.
sub / return — Define una función / devuelve su resultado.
close — Cierra una conexión o archivo.

Ejecutar la aplicación

6

Acciones

- Con hola_web.pl abierto, pulsa el botón verde Run (o Shift+F10).
- En la consola de IntelliJ aparece el mensaje del servidor.
- El programa queda «escuchando»: es normal, no se detiene solo.
- Para detenerlo, pulsa el botón rojo Stop.

Lo que verás en la consola

```
Servidor escuchando en  
  http://localhost:8080  
(pulsa Stop en IntelliJ  
 para detenerlo)
```

Ctrl + clic sobre la dirección <http://localhost:8080> para abrirla directo en el navegador.

Abrir y probar en el navegador

7

Acciones

- Abre `http://localhost:8080` en el navegador.
- Verás «¡Hola Mundo, Mundo!».
- Escribe tu nombre y pulsa Saludar.
- La URL cambia a `?nombre=...` y el saludo se personaliza.



La aplicación funcionando con HTML5 semántico

Dos ideas que debes recordar

Las dos reglas de oro del backend

- Validar TODA entrada en el servidor: el cliente es manipulable, nunca de fiar.
- Escapar TODA salida con `escape_html`: neutraliza etiquetas como `<script>` (anti-XSS).
- El servidor es la única frontera de confianza: ahí vive la lógica y la seguridad.

HTML5 semántico y moderno

- Estructura: `header`, `nav`, `main`, `article`, `section`, `aside`, `footer`.
- Elementos nuevos: `time`, `mark`, `details/summary`, `meter`, `output`.
- Formularios con validación nativa: `required`, `minlength`, `type=search`.
- Accesibilidad: `aria-label` y `:focus-visible`.

Solución de problemas comunes

«Can't locate CGI.pm»

Solo si usas use CGI. Strawberry ya lo incluye; si faltara: cpan CGI. Nuestro mini servidor NO lo necesita.

El programa «no hace nada»

Es normal: un servidor se queda escuchando. Abre el navegador en <http://localhost:8080>.

«Address already in use» / puerto ocupado

Otro proceso usa el 8080. Pulsa Stop, o cambia \$puerto a 8090 en el código.

IntelliJ no reconoce perl.exe

Revisa el Paso 3: el intérprete debe apuntar a C:\Strawberry\perl\bin\perl.exe.

Aviso de certificado al instalar plugins

Red de la UTEQ con inspección SSL: Save as Trusted, o instala el plugin desde disco.

¡Lo lograste!

Instalaste y configuraste el entorno (Strawberry Perl + IntelliJ + plugin Perl).

Creaste un programa PERL que actúa como servidor web sin necesidad de Apache.

Entendiste cada instrucción operador y función del código, no solo lo copiaste.

Aplicaste las dos reglas de oro: validar la entrada y escapar la salida.

Siguiente reto: añadir más campos al formulario y procesarlos, o pasar el saludo a PHP y comparar (Semana 5).